SHARED EQUIPMENT. PROTECTED REPAIR RESERVES.

Every rental funds its next repair.



A workshop ledger that separates rental income, refundable deposits, and the money that keeps each item working.

MAKERSPACES · CAMPUS HUBS · TOOL LIBRARIES

The tool is shared.
The repair budget usually isn’t.

Repeated use creates wear. The workshop needs a clear answer to three practical questions.

01

Who pays for the next repair?

Rental revenue can be spent before a tool breaks. Maintenance needs its own protected balance.

02

What happened to the deposit?

Damage claims need recorded evidence, clear approval rules, and a visible refund deadline.

03

Can members check the record?

Operators need a history that connects payments, returns, decisions, and repairs to an item.

ONE ITEM. ONE CONTINUING REPAIR RESERVE.

Scan. Rent. Return.
Keep the equipment in use.

01 / REGISTER

Give each tool a QR

Set its fee, deposit, reserve share, and distinct arbiter.

02 / RENT

Fund the next repair

The fee splits into operator income and this item’s reserve.

03 / RETURN

Record the evidence

A photo commitment starts the dispute window and Cronpot job.

04 / REFUND

Close the deposit

The keeper executes the due job; contract rules enforce eligibility.

05 / RESOLVE

Review a damage claim

The operator’s proposal needs renter or arbiter approval.

06 / REPAIR

Pay from the reserve

An arbiter-approved repair spends the item’s own repair funds.

THREE BALANCES. DIFFERENT PERMISSIONS.

A deposit is not income.
A repair reserve stays with its item.

OPERATOR INCOME

80%

Of the rental fee in the demo configuration. Withdrawable by the operator.

ITEM REPAIR RESERVE

20%

Of the rental fee in the demo configuration. Spent only on approved repairs.

REFUNDABLE DEPOSIT

Separate

Held as a rental liability. Returned in full or reduced by an approved claim.

The live camera rental used this split. Human assessment still determines damage; unresolved votes can delay settlement.

RUNTIME PRIMITIVES, WITH MEASURED BEHAVIOR

Solidity rules.
Native accounts and evidence.

PALLET-REVIVE + SUBSTRATE SIGNING

Use community wallets

Substrate-signed Revive calls run Solidity accounting and permissions. V3 funding automatically maps native accounts to H160.

14 NATIVE / 18 EVM DECIMALS

Keep value exact

The funded P4 probe measured a 10,000 conversion factor. Integer conversions reject fractional native-unit dust.

TRANSACTIONSTORAGE

Commit the final photo bytes

A signed 1 KiB store succeeded in P6. SHA-256 checks the displayed bytes; a persistent public backup remains outstanding.

CRONPOT + FINALIZED NATIVE EVENTS

Reusable timed execution

MeterMend integrates a generic timed-call registry that other contracts can reuse. SQLite retains finalized history beyond node pruning.

KEEPER MODE SELECTED ON V3

Automatic execution. Contract-enforced refund authority.

“refunds execute automatically via Cronpot; execution is permissionless and our keeper runs it.”

01

Return

MeterMend records the deadline and schedules a Cronpot job.

02

Wait

The configured keeper observes finalized chain time.

03

Execute

Any eligible caller can execute the due Cronpot job.

04

Refund

MeterMend checks status and deadline before releasing a deposit.

What the funded probe found: the native Scheduler rejected our signed origin with `BadOrigin` (P5, block 213576). It is not the refund executor. The original Cronpot callback ran out of gas; the keeper refunded 0.05 tPOT directly. Corrected Cronpot: 49 tests, not deployed.

METERMEND.VERCEL.APP/PROOF ↗

Live settlement and reserve proof. The failed callback stays visible.

Ping contract Finalized · 213570	`0x96ee88d272550530b518cbaa15a80cf5d38cdcfd` Runtime deployment and signed value movement verified	`tx 0x5003af97…24a011c` ↗
P4 / P6 probes Finalized	Exact 14/18 value movement; signed 1 KiB native store Blocks 213573 and 213579 · Full receipts retained	Funded evidence ↗
Cronpot Finalized · 265131	`0x41635a685448cfa86da1600656f103a211e398f4` Original deployment transaction finalized after the node resumed.	`tx 0xd60b3e13…1a0df93` ↗
MeterMend Finalized · 271379	`0x61382db168db646b899234cd707c7ba0142c4071` Direct refund 311559 · dispute 311952 · repair 311961. Income 311966 · rejected reserve spending 311970.	`tx 0x23b8aa6f…62b8f17` ↗

OPEN SOURCE · MIT ↗

Built to preserve the record when a connection fails.

49

Contract tests

Accounting, reserve isolation, dispute votes, callback failures, and fuzzed fee splits.

Corrected Cronpot included; CI passed November 3.

241

TypeScript tests

Chain handling, durable services, transaction recovery, web flows, and deployment scripts.

November 3 CI · 8253778 · package and script checks passed.

29

Browser checks

Desktop/mobile layout, wallet states, photo integrity, and recovery after reload.

Synthetic recovery fixtures; no payment was submitted.

Durable transaction hashes | Account locks across tabs | Offline retained history | Production build passes

PROPOSED MODEL · PILOT VALIDATION AHEAD

Earn when shared equipment is used and maintained.

PROPOSED REVENUE

A small service fee per completed rental.

Price for the workshop’s operating budget. Keep refundable deposits and protected repair money separate.

The current testnet contract does not collect a MeterMend platform fee. Pricing and collection are future product work.

Makerspaces

Shared tools, membership communities, and repair accountability.

Campus hubs

Student equipment, shared media kits, and transparent handovers.

Tool libraries

Repeated lending with a visible item-by-item maintenance history.

Make the next repair part of every rental.

NOW / SUBMISSION

Finish operations + video

Prove corrected Cronpot automation, host durable services, complete the physical phone rehearsal, and record the actual demo.

NEXT / PILOT

Learn in workshops

Test onboarding, dispute handling, reserve adequacy, evidence retention, and willingness to pay with real operators.

THEN / ECOSYSTEM

Reuse Cronpot

Document timed-call integration, improve keeper operations, and pursue DeBox as the target community integration.

Support needed: reliable testnet access, a persistent operations host, and pilot workshop partners.

metermend.vercel.app ↗

github.com/willy264/portal_dot ↗